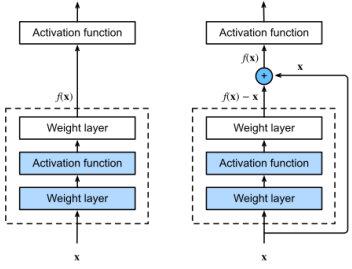
Introduction to Deep Learning

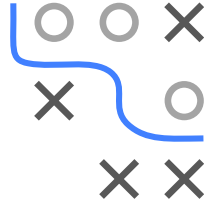
Convolutional Neural Networks II
Modern Architectures - II



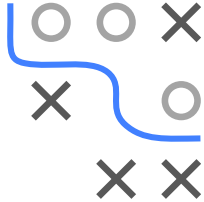
Learning goals

- GoogleNet
- ResNet
- DenseNet
- U-Net

GoogLeNet



INCEPTION MODULES I



- The Inception block is equivalent to a subnetwork with four paths.
- It extracts information in parallel through convolutional layers of different window shapes and max-pooling layers.
- 1×1 convolutions reduce channel dimensionality on a per-pixel level. Max-pooling reduces the resolution.

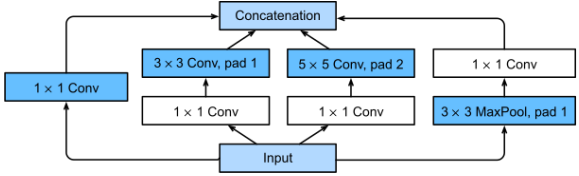
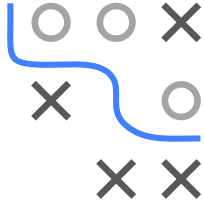


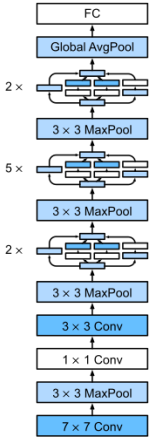
Figure: Inception Block (Dive into Deep Learning).

GOOGLENET ARCHITECTURE I

- GoogLeNet connects multiple well-designed Inception blocks with other layers in series.
- The ratio of the number of channels assigned in the Inception block is obtained through a large number of experiments on the ImageNet dataset.
- GoogLeNet, as well as its succeeding versions, was one of the most efficient models on ImageNet, providing similar test accuracy with lower computational complexity.



GOOGLENET ARCHITECTURE II



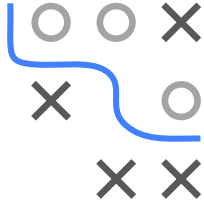
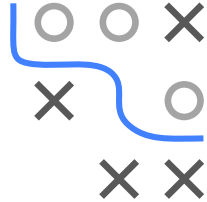
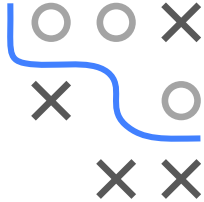


Figure: The GoogLeNet architecture (Dive into Deep Learning).

Residual Networks (ResNet)



RESIDUAL BLOCK (SKIP CONNECTIONS) I



Problem setting: theoretically, we could build infinitely deep architectures as the net should learn to pick the beneficial layers and skip those that do not improve the performance automatically.

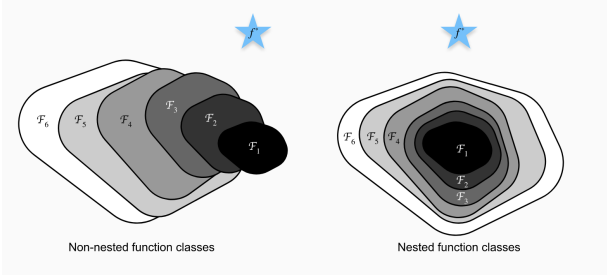
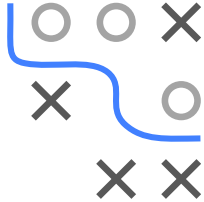


Figure: For non-nested function classes, a larger function class does not guarantee to get closer to the "truth" function ($\mathcal{F}*$). This does not happen in nested function classes (Dive into Deep Learning).

RESIDUAL BLOCK (SKIP CONNECTIONS) II



- But: this skipping would imply learning an identity mapping $\boldsymbol{x} = \mathcal{F}(\boldsymbol{x})$. It is very hard for a neural net to learn such a 1:1 mapping through the many non-linear activations in the architecture.
- Solution: offer the model explicitly the opportunity to skip certain layers if they are not useful.
- Introduced in *He et. al , 2015* and motivated by the observation that stacking evermore layers increases the test- as well as the train-error ($\neq$ overfitting).

RESIDUAL BLOCK (SKIP CONNECTIONS) III

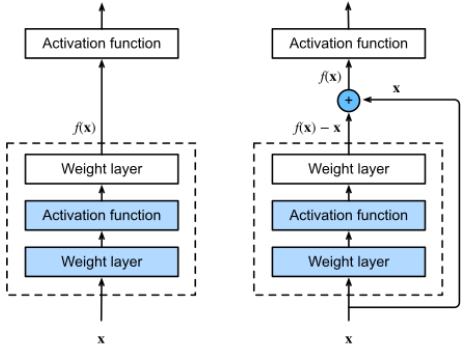
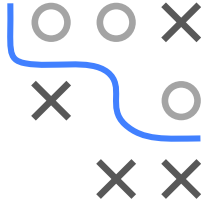
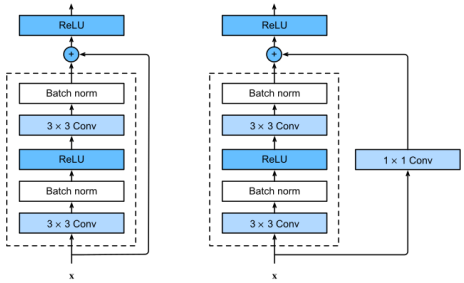


Figure: A regular block (left) and a residual block (right) (Dive into Deep Learning).



RESIDUAL BLOCK (SKIP CONNECTIONS) IV



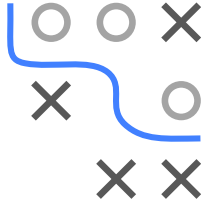
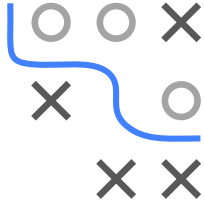


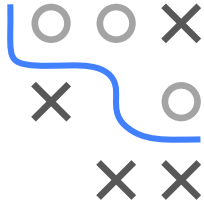
Figure: ResNet block with and without 1×1 convolution.The information flows through two layers and the identity function. Both streams of information are then element-wise summed and jointly activated (Dive into Deep Learning).

RESIDUAL BLOCK (SKIP CONNECTIONS) V



- Let $\mathcal{H}(\boldsymbol{x})$ be the optimal underlying mapping that should be learned by (parts of) the net.
- $\boldsymbol{x}$ is the input in layer *l* (can be raw data input or the output of a previous layer).
- $\mathcal{H}(\boldsymbol{x})$ is the output from layer *l*.
- Instead of fitting $\mathcal{H}(\boldsymbol{x})$, the net is ought to learn the residual mapping $\mathcal{F}(\boldsymbol{x}) := \mathcal{H}(\boldsymbol{x}) - \boldsymbol{x}$ whilst $\boldsymbol{x}$ is added via the identity mapping.
- Thus, $\mathcal{H}(\boldsymbol{x}) = \mathcal{F}(\boldsymbol{x}) + \boldsymbol{x}$, as formulated on the previous slide.
- The model should only learn the **residual mapping** $\mathcal{F}(\boldsymbol{x})$
- Thus, the procedure is also referred to as **Residual Learning**.

RESIDUAL BLOCK (SKIP CONNECTIONS) VI



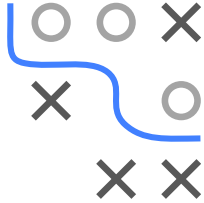
- The element-wise addition of the learned residuals $\mathcal{F}(\boldsymbol{x})$ and the identity-mapped data $\boldsymbol{x}$ requires both to have the same dimensions.
- To allow for downsampling within $\mathcal{F}(\boldsymbol{x})$ (via pooling or valid-padded convolutions), the authors introduce a linear projection layer W_s .
- W_s ensures that $\boldsymbol{x}$ is brought to the same dimensionality as $\mathcal{F}(\boldsymbol{x})$ such that:

$$y = \mathcal{F}(\boldsymbol{x}) + W_s\boldsymbol{x},$$

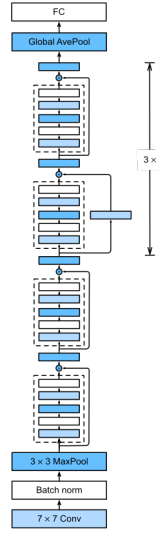
- y is the output of the skip module and W_s represents the weight matrix of the linear projection (# rows of W_s = dimensionality of $\mathcal{F}(\boldsymbol{x})$).
- This idea applies to fully connected layers as well as to convolutional layers.

RESNET ARCHITECTURE I

- The residual mapping can learn the identity function more easily, such as pushing parameters in the weight layer to zero.
- We can train an effective deep neural network by having residual blocks.
- Inputs can forward propagate faster through the residual connections across layers.
- ResNet had a major influence on the design of subsequent deep neural networks, both for convolutional and sequential nature.



RESNET ARCHITECTURE II



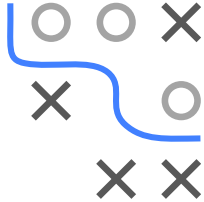
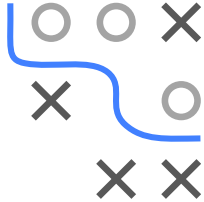
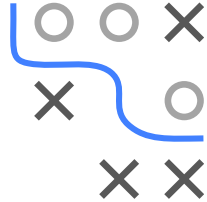


Figure: The ResNet-18 architecture (Dive into Deep Learning).

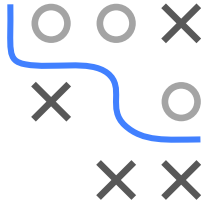
RESNET ARCHITECTURE III



Densely Connected Networks (DenseNet)

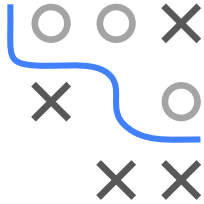


FROM RESNET TO DENSENET I



- ResNet significantly changed the view of how to parametrize the functions in deep networks.
- DenseNet (dense convolutional network) is to some extent the logical extension of this [Huang et al., 2017].
- Dense blocks where each layer is connected to every other layer in feedforward fashion.
- Alleviates vanishing gradient, strengthens feature propagation, encourages feature reuse.
- To understand how to arrive at it, let us take a small detour to mathematics:
  - Recall the Taylor expansion for functions. For the point $x = 0$ it can be written as:
    $f(x) = f(0) + f'(0)x + \frac{f''(0)}{2!}x^2 + \frac{f'''(0)}{3!}x^3 + \dots$

FROM RESNET TO DENSENET II



- The key point is that it decomposes a function into increasingly higher order terms. In a similar vein, ResNet decomposes functions into : $f(\mathbf{x}) = \mathbf{x} + g(\mathbf{x})$.
- That is, ResNet decomposes f into a simple linear term and a more complex nonlinear one. What if we want to capture (not necessarily add) information beyond two terms? One solution was DenseNet [Huang et al., 2017].

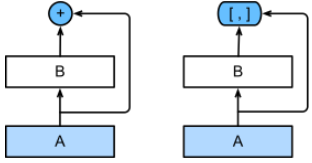
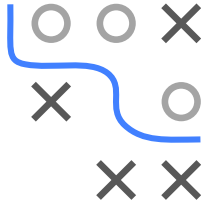


Figure: DensNet Block (Dive into Deep Learning).

FROM RESNET TO DENSENET III



As shown in previous Figure, the key difference between ResNet and DenseNet is that in the latter case outputs are concatenated (denoted by $[,]$) rather than added. As a result, we perform a mapping from x to its values after applying an increasingly complex sequence of functions:

$$\mathbf{x} \to [\mathbf{x}, f_1(\mathbf{x}), f_2([\mathbf{x}, f_1(\mathbf{x})]), f_3([\mathbf{x}, f_1(\mathbf{x}), f_2([\mathbf{x}, f_1(\mathbf{x})])]), \ldots] .$$

In the end, all these functions are combined in MLP to reduce the number of features again. In terms of implementation this is quite simple: rather than adding terms, we concatenate them.
The name DenseNet arises from the fact that the dependency graph between variables becomes quite dense. The last layer of such a chain is densely connected to all previous layers.

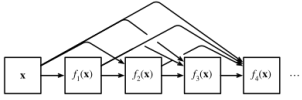
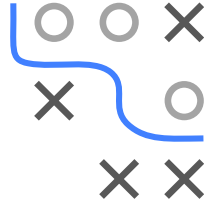


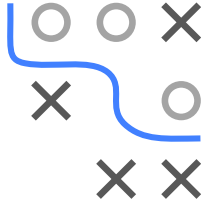
Figure: The DensNet architecture (Dive into Deep Learning).

U-Net

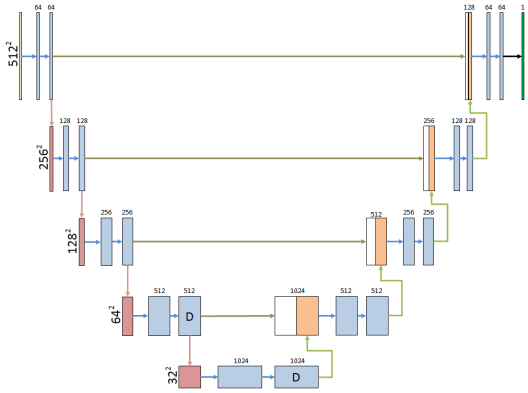


U-NET I

- U-Net is a fully convolutional net that makes use of upsampling (via transposed convolutions, for example) as well as skip connections.
- Input images are getting convolved and down-sampled in the first half of the architecture.
- Then, they are getting upsampled and convolved again in the second half to get back to the input dimension.
- Skip connections throughout the net combine feature maps from earlier layers with those from later layers by concatenating both sets of maps along the depth/channel axis.
- Only convolutional and no dense layers are used.



U-NET II



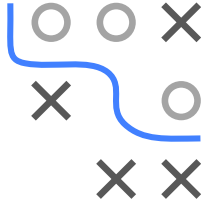
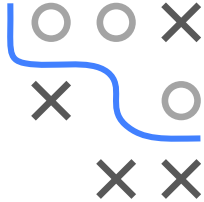


Figure: Illustration of the architecture. Blue arrows are convolutions, red arrows max-pooling operations, green arrows upsampling steps and the brown arrows merge layers with skip connections. The height and width of the feature blocks are shown on the vertical and the depth on the horizontal. D are dropout layers.

U-NET III



U-NET I

- Example problem setting: train a neural net to pixelwise segment roads in satellite imagery.
- Answer the question: **Where is the road map?**

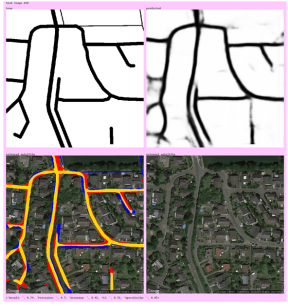
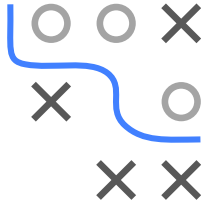
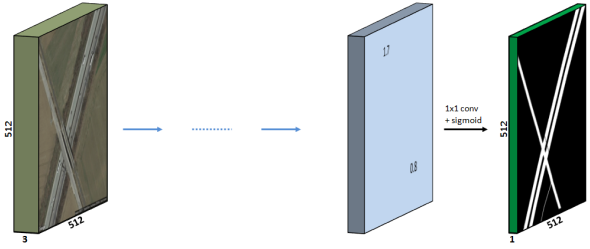


Figure: Model prediction on a test satellite image. Yellow are correctly identified pixels, blue false negatives and red false positives.

U-NET II

- The net takes an RGB image [512, 512, 3] and outputs a binary (road / no road) probability mask [512, 512, 1] for each pixel.
- The model is trained via a binary cross entropy loss which was combined over each pixel.





$$Output_{i,j} = \frac{1}{1+exp(-1.7 \cdot \mathbf{w})} \in (0,1)$$

Figure: Scheme for the input/ output of the net architecture.

REFERENCES I

Dive into Deep Learning . (n.d.-a). *8.4. Multi-branch networks (googlenet)*. `http://d2l.ai/chapter_convolutional-modern/googlenet.html`

Dive into Deep Learning . (n.d.-b). *8.6. Residual Networks (ResNet) and ResNeXt*. `http://d2l.ai/chapter_convolutional-modern/resnet.html`

Dive into Deep Learning . (n.d.-c). *8.7. Densely Connected Networks (DenseNet)* . `http://d2l.ai/chapter_convolutional-modern/densenet.html`

